Supabase Actual Schema

1. Core Application Schema (public)

Tables & Structure

```
-- Organizations (Tenant isolation)
organizations {
  uuid id PK
  text name
  uuid owner_id FK -> auth.users.id
  timestamptz created_at
  text api_key (unique)
  jsonb metadata
}

-- AI Agents
agents {
  uuid id PK
  uuid organization_id FK -> organizations.id
  text name
  text description
  text status (active/inactive/blocked)
  text source (n8n/zapier)
  jsonb metadata
  text risk_profile (low/medium/high)
  text blocked_reason
  boolean verified
  jsonb scope
  timestamptz created_at
}

-- Governance Policies
policies {
  uuid id PK
  uuid organization_id FK -> organizations.id
  text name
  text description
  jsonb rule (action/conditions)
  boolean is_enabled
  timestamptz created_at
}

-- Audit Ledger (Immutable events)
ledger_events {
  uuid id PK
  uuid agent_id FK -> agents.id
  text action
  text status (allow/pause/block)
  jsonb metadata
  timestamptz created_at
  uuid organization_id FK -> organizations.id
  text prev_hash (blockchain-style chaining)
  text action_type
```

```
}

-- Activity Logs
logs {
  uuid id PK
  timestamptz created_at
  text agent
  text type (DB_QUERY/API_CALL/ANALYSIS)
  text intent
  integer gate_ms
  text status (allowed/blocked/paused)
  text company_id
}
```

Relationships

- **Organizations** 1:N **Agents** (Cascade delete)
- **Organizations** 1:N **Policies** (Cascade delete)
- **Organizations** 1:N **Ledger_Events**
- **Agents** 1:N **Ledger_Events** (Cascade delete)
- **Auth.Users** 1:N **Organizations** (Owner relationship, Cascade delete)

Constraints

- `agents_risk_profile_check`: ENUM ('low', 'medium', 'high')
- `agents_source_check`: ENUM ('n8n', 'zapier')
- `agents_status_check`: ENUM ('active', 'inactive', 'blocked')
- `ledger_events_status_check`: ENUM ('allow', 'pause', 'block')
- Unique index on `organizations(api_key)`

---

2. Entity Relationship Diagram

```
erDiagram
    auth_users ||--o{ organizations : owns
    auth_users {
        uuid id PK
        varchar email
        varchar encrypted_password
        jsonb raw_app_meta_data
        jsonb raw_user_meta_data
    }

    organizations ||--o{ agents : contains
    organizations ||--o{ policies : defines
    organizations ||--o{ ledger_events : generates
    organizations {
        uuid id PK
        string name
        uuid owner_id FK
        string api_key UK
        jsonb metadata
        timestamp created_at
    }

    agents ||--o{ ledger_events : logs
```

```
    agents {
        uuid id PK
        uuid organization_id FK
        string name
        string status
        string source
        string risk_profile
        boolean verified
        jsonb metadata
        jsonb scope
        timestamp created_at
    }

    policies {
        uuid id PK
        uuid organization_id FK
        string name
        jsonb rule
        boolean is_enabled
        timestamp created_at
    }

    ledger_events {
        uuid id PK
        uuid agent_id FK
        uuid organization_id FK
        string action
        string status
        string prev_hash
        jsonb metadata
        timestamp created_at
    }

    logs {
        uuid id PK
        string agent
        string type
        string intent
        string status
        integer gate_ms
        timestamp created_at
    }
```

---

3. Sample Data Extracts

Organizations

```
INSERT INTO public.organizations (id, name, owner_id, created_at, api_key, metadata)
VALUES (
  'bb68d152-08a4-4af9-8733-6b3124119af5',
  'Test Org',
  'a4787916-3fd2-46ed-9ec4-ddf646bdc4e9',
  '2026-03-17 17:01:20.045089+00',
  'whk_live_5ac1f9d6e1f24c51a5c3f2b4b1f0a9d2',
  '{}'
);
```

Agents

```
INSERT INTO public.agents (id, organization_id, name, status, created_at, source, metadata, r
isk_profile, verified)
VALUES
  ('24f8fafb-5326-4053-a14f-32a00dfd4a26', 'bb68d152-08a4-4af9-8733-6b3124119af5', 'Finance A
gent', 'active', '2026-03-20 18:10:38.570965+00', 'n8n', '{}', 'high', false),
  ('76372ad5-061d-4c05-8256-388636fc2fb8', 'bb68d152-08a4-4af9-8733-6b3124119af5', 'test-agen
t', 'active', '2026-03-18 02:09:59.669036+00', 'n8n', '{"last_seen": "2026-03-21T18:37:26.058
730+00:00"}', 'low', false),
  ('d88bcc8a-ed54-4638-81b9-4008e0f21080', 'bb68d152-08a4-4af9-8733-6b3124119af5', 'Webhook A
gent', 'active', '2026-03-21 19:47:36.810738+00', 'n8n', '{"last_seen": "2026-03-21T19:58:41.
154411+00:00"}', 'low', false);
```

Ledger Events (Audit Trail)

```
INSERT INTO public.ledger_events (id, agent_id, action, status, metadata, created_at, organiz
ation_id)
VALUES
  ('947787cc-b261-4323-9c43-f57e43d0eb23', '76372ad5-061d-4c05-8256-388636fc2fb8', 'retrieve
pending claims for daily triage', 'allow', '{"source": "n8n"}', '2026-03-18 02:10:00.14635+0
0', 'bb68d152-08a4-4af9-8733-6b3124119af5'),
  ('080ad4bc-7c34-44db-a132-dc645cdc3062', '76372ad5-061d-4c05-8256-388636fc2fb8', 'retrieve
pending claims for daily triage', 'block', '{"source": "n8n"}', '2026-03-18 04:24:05.630924+0
0', 'bb68d152-08a4-4af9-8733-6b3124119af5'),
  ('ba78d0c8-b0c3-443d-ad62-7ead0a3dbe08', '76372ad5-061d-4c05-8256-388636fc2fb8', 'retrieve
pending claims for daily triage', 'pause', '{"source": "n8n"}', '2026-03-18 04:28:19.047547+0
0', 'bb68d152-08a4-4af9-8733-6b3124119af5');
```

Policies

```
INSERT INTO public.policies (id, organization_id, name, description, rule, is_enabled, create
d_at)
VALUES (
  '6c308597-8165-4c84-8ec8-11af1955dc58',
  'bb68d152-08a4-4af9-8733-6b3124119af5',
  'Review retrieve',
  NULL,
  '{"action": "review", "condition": "retrieve"}',
  true,
  '2026-03-18 04:26:50.933306+00'
);
```

---

4. Row Level Security (RLS) Policies

Agents Table

Policy	Operation	Definition
`agents_owner_select`	SELECT	User must own the parent organization
`agents_owner_insert`	INSERT	User must own the target organization
`agents_owner_update`	UPDATE	User must own the parent organization
`agents_owner_delete`	DELETE	User must own the parent organization
`agents_select_own_org`	SELECT	org_id IN (SELECT orgs where owner_id = auth.uid())
`agents_insert_own_org`	INSERT	org_id IN (SELECT orgs where owner_id = auth.uid())

Policy	Operation	Definition
`agents_update_own_org`	UPDATE	org_id IN (SELECT orgs where owner_id = auth.uid())
`agents_delete_own_org`	DELETE	org_id IN (SELECT orgs where owner_id = auth.uid())

Organizations Table

Policy	Operation	Definition
`org_owner_select`	SELECT	owner_id = auth.uid()
`org_owner_insert`	INSERT	owner_id = auth.uid()
`org_owner_update`	UPDATE	owner_id = auth.uid()
`org_owner_delete`	DELETE	owner_id = auth.uid()

Policies Table (Governance Rules)

Policy	Operation	Definition
`policies_owner_select`	SELECT	EXISTS (SELECT 1 FROM organizations o WHERE o.id = policies.organization_id AND o.owner_id = auth.uid())
`policies_owner_insert`	INSERT	Same as above
`policies_owner_update`	UPDATE	Same as above
`policies_owner_delete`	DELETE	Same as above

Ledger Events Table

Policy	Operation	Definition
`ledger_owner_select`	SELECT	EXISTS (SELECT 1 FROM agents a JOIN organizations o ON o.id = a.organization_id WHERE a.id = ledger_events.agent_id AND o.owner_id = auth.uid())
`ledger_owner_insert`	INSERT	Same validation via agent→organization chain
`ledger_events_select_own_org`	SELECT	organization_id IN (SELECT orgs where owner_id = auth.uid())
`ledger_events_insert_own_org`	INSERT	organization_id IN (SELECT orgs where owner_id = auth.uid())

---

5. Key Observations

Architecture Patterns

1. **Supabase-native Architecture**: Heavy use of `auth.users`, `auth.identities`, and Supabase-specific schemas (realtime, storage, vault)
2. **Multi-tenancy**: Strict RLS policies enforce organization-based data isolation; no cross-tenant data access possible
3. **Immutable Audit Trail**: The `ledger_events` table includes `prev_hash`, suggesting a blockchain-style or Merkle-chain audit pattern for tamper-evident logging
4. **Agent Governance**: Risk profiling (`low`/`medium`/`high`) combined with policy rules indicates this is an **AI Agent oversight/governance platform**

Security Features

- **API Key Isolation**: Organizations have unique API keys (`whk_live_*` format suggests webhook integration)
- **JWT-based RLS**: All policies rely on `auth.uid()` from JWT claims
- **Source Validation**: Agents restricted to specific integration sources (`n8n`, `zapier`)
- **Status Enforcement**: Database-level constraints prevent invalid state transitions (e.g., blocking without reason)

Data Integrity

- **Foreign Key Cascading**: Deleting an organization cascades to agents, policies, and ledger events
- **Unique Constraints**: API keys and OAuth identifiers have unique indexes
- **Generated Columns**: `confirmed_at` in auth.users is generated from LEAST(email_confirmed_at, phone_confirmed_at)

Performance Considerations

- **Indexing Strategy**: Composite indexes on `(organization_id, created_at)` for ledger events and `(bucket_id, name)` for storage
- **Partitioning**: `realtime.messages` is partitioned by range on `inserted_at`
- **Search Optimization**: Storage schema includes optimized `search_v2` function with collation-specific indexing for object listing

Notable Extensions

- **pg_graphql**: GraphQL API support enabled
- **pg_stat_statements**: Query performance monitoring
- **pgcrypto**: Cryptographic functions for secure data handling
- **uuid-ossp**: UUID generation (though `gen_random_uuid()` is used, likely from pgcrypto)

This schema represents a **secure, multi-tenant AI Agent management platform** with emphasis on audit trails, policy enforcement, and integration with workflow automation tools (n8n/Zapier).